PROJECT SUBMISSION

Causa

Broken data or real change? Know what broke, what it cost, and who saw it.

A root-cause engine for business metrics, built on Exasol. By Shweta Rani.

Public code repository	https://github.com/rani700/causa
Live demo (guided tour)	https://causa.codeshare.co.in/?tour=1
Demo video (2:52)	https://github.com/rani700/causa/releases/download/demo-video/Causa_demo_3min.mp4
Licence	MIT (LICENSE)

Inside this file

1. Problem statement: who has the problem and why it matters
2. Pitch deck (11 slides)
3. Demo video
4. README: overview, setup and usage
5. Deployment instructions and run guide
6. Licence

1 · PROBLEM STATEMENT

Problem statement

Who has the problem

Anyone who reports a business metric and has to stand behind it:

- **Analysts** who are asked "why did revenue drop last week?" on a Monday morning.
- **Data engineers** who own the pipelines and get pulled in whenever a number looks odd.
- **Finance and operations leaders** who put the number in a board pack, a forecast or a month-end close, and make decisions on it.

What happens today

When a metric moves, two very different things can be behind it: the business changed, or the data broke. A feed stalled after a schema change, a load ran twice, a currency rate was wrong, a batch arrived late.

Telling them apart is slow, manual work. An analyst opens tables, compares row counts, reads pipeline logs and asks other teams, often for a day or more. The tools in place each answer half of the question:

- **Data observability tools** stop at "this table looks wrong". They cannot say what that means for revenue.
- **BI explain features** break a change down by segment, but assume the data is right.

Nobody joins the two, so even after the checking, three questions usually stay open.

Why it matters

By the time someone finds the broken load, the wrong number has already travelled. It was on the revenue dashboard, in the Monday board pack, in a forecast, in the scores an ML model synced to Salesforce. Decisions were made on broken data, and each such episode makes people trust the numbers a little less. Fixing the table does not fix any of that: someone has to work out how wrong each number was and tell everyone who used it.

What Causa answers

For any metric and period, Causa answers the three open questions directly:

1. **How much of the change is broken data, in euros?** It finds the data incidents behind the metric and measures each one's exact effect by re-running the company's own logged pipeline SQL on repaired data inside Exasol.
2. **What is the real business change?** What remains once the incidents are removed, judged against normal variation and explained by segment (region, country, customer).

3. **Who already used the wrong numbers?** Every dashboard, scheduled report, ML model, reverse-ETL sync and analyst that read affected data in the exposure window, each with a drafted correction notice.

It needs only read access to the warehouse and its query log: no instrumentation, no pipeline changes, no hand-drawn lineage. On a benchmark of 20 randomly drawn simulated companies it raised 100% of data incidents as alerts and left 97% of business changes calm, and 218 of 220 verdicts put the data impact within 10% of the change (benchmark).

Live demo: https://causa.codeshare.co.in (guided tour: https://causa.codeshare.co.in/?tour=1).

Causa

Broken data or real change?
Know what broke, what it cost, and who saw it.

A root-cause engine for business metrics, built on Exasol

Shweta Rani · causa.codeshare.co.in · github.com/rani700/causa

THE PROBLEM

Monday, 9 a.m. The revenue dashboard says sales are down 34%.

Is the business in trouble?

Customers buying less, a market slowing down, a real loss to act on.

Or did the data break?

A feed stalled after a schema change, a load ran twice, a currency rate was wrong.

Finding out means opening tables, comparing row counts, reading pipeline logs and asking other teams: hours, often days.

WHO HAS IT, AND WHY IT MATTERS

By the time someone finds the broken load, the wrong number has already travelled.

Analysts

asked “why did revenue drop?” and expected to answer today

Data engineers

pulled in whenever a number looks odd, to dig through pipelines

Leaders

who put the number in a board pack or forecast and decide on it

It was on the dashboard, in the board pack, in a forecast, in scores synced to Salesforce. Fixing the table fixes none of that, and every episode costs trust in the numbers.

THE GAP

Today’s tools each answer half the question.

Data observability

says “this table looks wrong”, but not what that means for revenue

BI “explain” features

break a change down by segment, but assume the data is right

Nobody joins the two, so three questions stay open

How much of the change is broken data? · What is the real change? · Who already used the wrong number?

WHAT CAUSA ANSWERS

The same Monday, answered in seconds: revenue −€13.68M week on week.

1 · BROKEN DATA

−€11.69M

a schema change emptied a timestamp in the EMEA feed; measured by replaying the pipeline, 95% confidence

2 · REAL CHANGE

−€1.99M

-4.9%, within normal variation, with the segments that drove it

3 · WHO SAW IT

Every reader

board pack, 3 dashboards, a churn model and its Salesforce sync, 3 analysts, each with a correction notice

It needs only read access to the warehouse and its query log: no instrumentation, no pipeline changes, no hand-drawn lineage.

HOW IT WORKS

Five engines, one chain of evidence

1

Lineage

column-level map built from the query log with sqlglot, plus the tools behind each session

2

Health

zero-config monitors: nulls, row loss, duplicates, schema changes, re-runs

3

Attribution

GROUPING SETS drill-down, volume vs rate split, significance vs past weeks

4

Verdict

links each incident to the metric and weighs the evidence: real or broken

5

Replay & recall

re-runs the logged SQL on repaired data, then finds every reader

On top: a web UI, HTTP API, CLI, an AI agent where every figure comes from a tool, an MCP server for Claude Desktop and Claude Code, and a watcher that announces new incidents.

WHY EXASOL

The warehouse does the heavy lifting, and the data never leaves it.

1

Lineage for free

EXA_DBA_AUDIT_SQL records every statement and who ran it, so the pipeline map needs no setup

2

Replays as real SQL

about 30 pipeline statements re-run per incident, inside Exasol, where the data lives

3

Pushed-down analytics

an explanation is 3 to 8 GROUPING SETS queries; a health scan is one query per table

DOES IT WORK?

Scored against ground truth on 20 randomly generated companies

120 / 120

data incidents raised as alerts

218 / 220

verdicts within 10% of the change, median error 0.0%

97%

of real business changes left calm

93%

of alerts match a real incident

On Exasol SaaS

same results as the local engine; a first investigation about 15 s, a repeated one instant

Honest about misses

both misses and every false alarm are listed in the benchmark report

BUILT TO RUN, NOT JUST TO DEMO

Live today, used the way people already work

Many ways in

Web UI with a guided tour, HTTP API, CLI, an Ask tab in plain English, and an MCP server so Claude Desktop and Claude Code can investigate directly

Production basics

Container built by CI on every push, deployed with GitOps; health and readiness probes; failover to a built-in copy while the warehouse is away

Always watching

A watcher announces each new serious incident once, with an optional Slack alert and a daily brief

Safe and cheap

Reads company data only; an AI answer costs under half a cent; 121 tests, lint and type checks clean

WHAT'S NEXT

From finding the damage to closing the loop

Deliver the corrections

send each notice by email or Slack and track who acknowledged it

More warehouses

Snowflake and BigQuery, whose query histories carry the same information

Ratio metrics

conversion rate, average order value and other metrics built from two numbers

Column-level blast radius

show exactly which columns of which reports an incident touched

Don’t just tell me the data is broken.
Tell me what broke, what it cost, and who saw it.

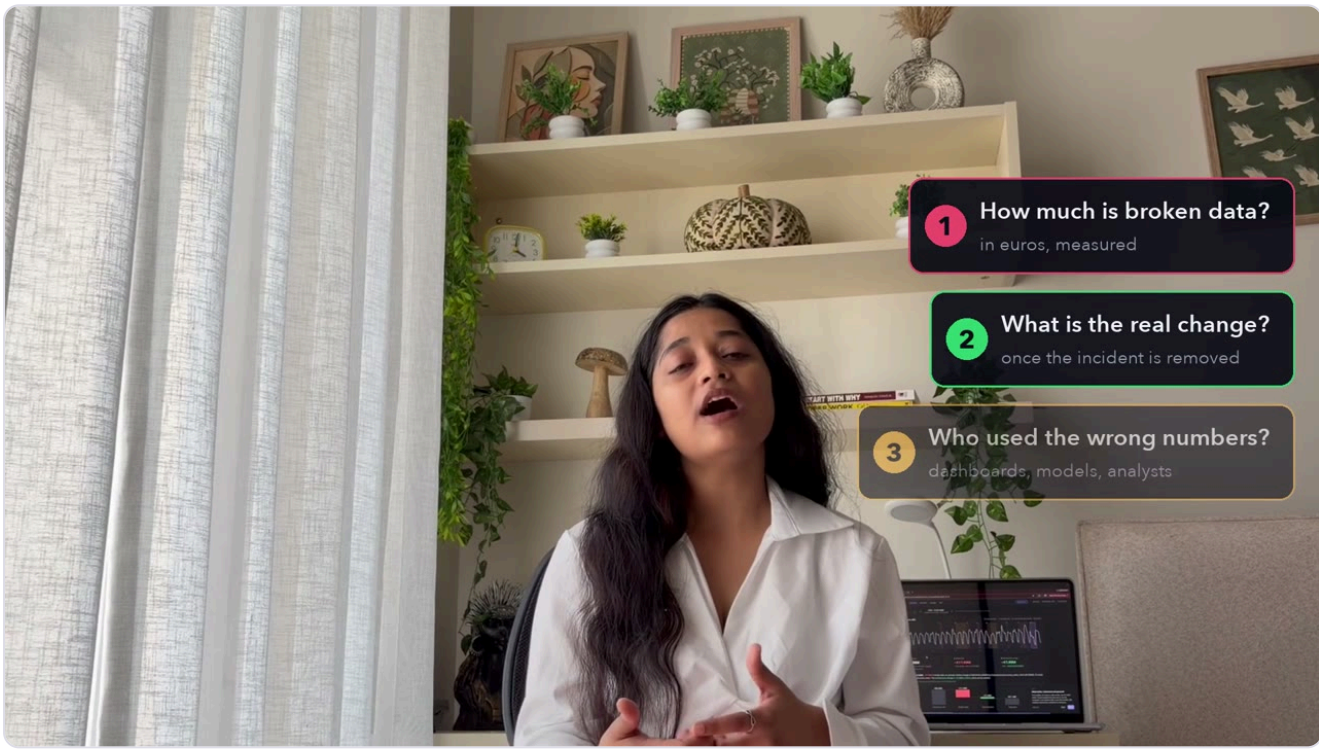
Try the live demo	`causa.codeshare.co.in/?tour=1`
Read the code	`github.com/rani700/causa`

Shweta Rani · MIT licence

3 · DEMO VIDEO

Demo video (2 minutes 52 seconds)

Watch or download: https://github.com/rani700/causa/releases/download/demo-video/Causa_demo_3min.mp4



What it shows, in order: the problem and the three questions Causa answers; the dashboard splitting a €13.68M drop into €11.69M of broken data and a −€1.99M real change; the Ask tab answering in plain English and listing who saw the wrong numbers; Causa inside Claude Desktop over MCP; how it is built; why Exasol; and the benchmark on 20 simulated companies.

README

A root-cause engine for business metrics, built on Exasol.

When a KPI moves, Causa answers the three questions that usually take a data team days:

1. **Is it real, or did the data break?** It splits the change into data incidents and real business change, and measures each incident's exact effect by re-running the company's own pipeline on repaired data.
2. **Why did it move?** Which segments drove the real part of the change.
3. **Who consumed the wrong number?** Every dashboard, report, ML model and sync that read affected data, with a correction notice for each.

Data observability tools flag broken tables but can't say what that means for revenue. BI "explain" features break revenue down but assume the data is correct. Causa joins the two, from nothing more than read access to the warehouse and its query log.

- **Live demo:** https://causa.codeshare.co.in (guided tour: https://causa.codeshare.co.in/?tour=1). It runs on Exasol SaaS, and on a built-in copy of the same data while the trial cluster is asleep.
- **Problem statement:** docs/problem-statement.md
- **Deployment and run guide:** docs/deployment.md
- **Demo video (3 min):** https://github.com/rani700/causa/releases/download/demo-video/Causa_demo_3min.mp4
- **Pitch deck and full write-up, one PDF:** Causa_submission.pdf

How well it works

Scored against ground truth on 20 randomly drawn companies (120 data incidents, 100 business changes):

Data incidents raised as alerts	**100%**
Business changes left calm (not alerted as data problems)	**97%**
Alerts that match a real incident	**93%**
Verdicts whose data impact is within 10% of the change	**218 / 220** (median error 0.0%)

Both misses (a big promotion partly read as a data problem; a late batch whose missing order lines were not recognised as healed) and every false alarm are listed in the report. The simulator is a benchmark: these numbers say the engines work on these kinds of incidents, not that they will on every warehouse. Reproduce with `uv run causa eval bench --seeds 20`.

Status

Lineage from the query log (column-level, row-gating predicates, blast radius)	✅
Data health detection, lineage-focused, grouped into cases	✅
Metric attribution (drill-down, volume/rate split, significance, restatements)	✅
Business-vs-data verdict with evidence and confidence	✅
Counterfactual replay of the real pipeline, recall with correction notices	✅
Web UI with a guided tour; HTTP API; CLI	✅
Investigator agent on Claude or OpenAI (`gpt-5-nano`), with cost per answer	✅
MCP server for Claude Desktop / Claude Code	✅
Watch: each new serious incident announced once, daily brief, Slack	✅
`causa init` : metric definitions proposed for a real warehouse	✅
Production: failover to a local copy, live refresh, health/readiness probes, container image	✅
Validated end to end on Exasol SaaS (2025.1); benchmark on 20 random companies	✅

Setup

Prerequisites: Python 3.12 or newer and uv. Docker only if you want a local Exasol (`make exasol-up`). No Exasol is needed to try Causa: by default it runs on a built-in DuckDB copy of the demo company.

```
git clone https://github.com/rani700/causa.git && cd causa
uv sync                              # install (or: make install)
uv run causa sim run                 # build the demo company in data/causa.duckdb (~70s)
make test                            # uv run pytest -q
make lint                            # ruff, ruff format --check, mypy
```

Usage

Web UI and API

```
uv run causa serve                   # web UI at http://127.0.0.1:8000, API docs at /docs
```

Open http://127.0.0.1:8000/?tour=1 for the guided tour. The API serves `/metrics` , `/metrics/{m}/explain|verdict|recall|brief` , `/health/cases` , `/lineage/{table}` and `POST /ask` ; `/health` and `/ready` are the liveness and readiness probes.

CLI

`uv run causa --help` lists every command. On the demo company:

```
uv run causa sim scenario          # the business events and data incidents that will be
injected
uv run causa lineage trace RPT.V_EXEC_REVENUE_DAILY --column NET_REVENUE_EUR
uv run causa lineage impact RAW.SHOP_ORDERS --since 2026-10-06 --until 2026-10-12
uv run causa health scan --since 2026-05-25 --until 2026-10-17
uv run causa eval health           # score the health engine against ground truth
uv run causa explain net_revenue --current 2026-10-06..2026-10-12
uv run causa explain net_revenue --current 2026-10-01..2026-10-05     # a churned whale
under a normal total
uv run causa metrics restatements net_revenue --since 2026-05-01 --until 2026-10-17 --
dimension REGION
uv run causa verdict net_revenue --current 2026-10-01..2026-10-07    # data incidents + the
real story
uv run causa eval verdict          # data impact (replayed) vs the true gap, for every event
uv run causa recall net_revenue --current 2026-10-06..2026-10-12 --out notices.md
uv run causa brief net_revenue --day 2026-10-08          # add --post with
CAUSA_SLACK_WEBHOOK_URL set
uv run causa ask "Why did revenue drop last week? Is it real?"   # needs OPENAI_API_KEY or
ANTHROPIC_API_KEY in .env
uv run causa mcp                   # MCP server (stdio) for Claude Desktop / Claude Code
uv run causa watch --once          # announce new serious incidents (Slack with
CAUSA_SLACK_WEBHOOK_URL)
uv run causa eval bench --seeds 20 # score both engines on 20 randomly drawn companies
```

MCP: Claude Desktop and Claude Code (no API key)

`causa mcp` serves the agent's tools over the Model Context Protocol. The client's own Claude does the investigating, on the user's Claude plan, and every figure still comes from Causa's engines. The tools are the same definitions the agent uses (marked read-only), the server tells the client how to investigate (start with `judge_change`, never compute figures, keep data incidents and business change apart), and the warehouse connection opens on the first tool call. Backend and credentials come from `.env`, as for every other command.

Claude Code: the repository's `.mcp.json` registers it; start `claude` in the repository and approve the `causa` server. Then ask, for example, *"Use causa: why did net revenue move in the week of 2026-10-06, and who saw wrong numbers?"*

Claude Desktop: Settings → Developer → Edit Config, add the server, and restart Claude Desktop:

```
{
  "mcpServers": {
    "causa": {
      "command": "/absolute/path/to/uv",
      "args": ["--directory", "/absolute/path/to/causa", "run", "causa", "mcp"]
    }
  }
}
```

Use absolute paths (`which uv`): Claude Desktop does not start from your shell. Add `"env": {"CAUSA_BACKEND": "duckdb"}` to use the local copy instead of Exasol. A DuckDB file can be open in only one process, so stop `causa serve` while Claude uses it (Exasol has no such limit).

Running on Exasol

Against Exasol

Exasol SaaS (free trial at https://cloud.exasol.com/signup): create a database, add your IPv4 to **Security → allow list**, and take the connection string, user name and a personal access token from **Connect via tools**. Or run `make exasol-up` for a local docker-db (x86_64 hosts).

```
cp .env.example .env                # CAUSA_BACKEND=exasol, DSN host:8563, user, token as password
uv run causa sim run
uv run causa eval health && uv run causa eval verdict
uv run causa lineage trace MART.DAILY_REVENUE --column NET_REVENUE_EUR --query-log audit
```

`--query-log audit` reads Exasol's own `EXA_STATISTICS.EXA_DBA_AUDIT_SQL` (auditing is on by default on SaaS; needs a DBA or `SELECT ANY DICTIONARY`): no instrumentation of the pipeline is needed.

Validated on an XSmall Exasol SaaS cluster (2025.1, eu-central-1), driven from a laptop in India:

	DuckDB	Exasol SaaS
Warehouse after 168 days (order lines / true revenue / query log / snapshots)	387,750 / €829,668,345.48 / 4,598 / 139,440	identical (reported revenue within €0.84 of €818M: exact DECIMAL arithmetic)
Health engine: incident recall / business calm / alert precision	100% / 100% / 100%	100% / 100% / 100% (21 s)
Verdict with replay: windows within tolerance, worst error	11/11, 0.8%	11/11, 0.8% (3 min)
Lineage of `NET_REVENUE_EUR` from the simulated log vs Exasol's audit log		identical columns and gating predicates

The simulator itself takes ~35 min there (one ~0.2 s round trip per statement from India to Frankfurt; small batches go through a prepared `INSERT`, large ones through `IMPORT`).

All SQL is authored in the **Exasol dialect**. The DuckDB backend exists only so the engines can be developed and tested without a cluster; it transpiles statements with `sqlglot`.

On your own Exasol

```
cp .env.example .env                # CAUSA_BACKEND=exasol, DSN, user, personal access token
uv run causa init                   # proposes causa.yaml: metrics, dimensions; checks the
audit log
uv run causa serve --query-log audit
```

Causa needs a read-only user that can read the tables behind the metrics and Exasol's audit log (`SELECT ANY DICTIONARY`; auditing is on by default on SaaS). It writes only to its own schemas: `CAUSA` (metric snapshots, notifications) and `CAUSA_REPLAY` (scratch tables of a replay, dropped after each run). Nothing in the pipeline has to change. The permissions are listed in docs/deployment.md.

Running it in production

Step-by-step instructions (local, Exasol SaaS, Docker, Kubernetes with Argo CD) are in docs/deployment.md. In short:

The image `ghcr.io/rani700/causa` is built by CI on every push to `main` (see `.github/workflows/`); it serves the web UI and API on port 8000 and bakes in the demo company as the fallback copy.

- **Failover.** With `CAUSA_FALLBACK_DUCKDB_PATH` set, a lost Exasol connection (never an SQL error) switches to a local DuckDB copy, retried every two minutes; the UI says so. Lineage, health and cached answers are rebuilt on whichever backend is active.
- **Fresh data.** Every five minutes Causa checks the query log's newest statement; new activity reloads lineage and health and drops cached answers.
- **Fast on a remote warehouse.** Each incident is replayed once and reused by every week it touches; answers are cached per question; the server prepares every incident's investigation at start.
- **Cost per decision.** Every investigation reports the warehouse statements and database seconds it took (or that it came from cache), every agent answer its token cost in dollars.
- **Probes.** `GET /health` (process up) and `GET /ready` (which warehouse answers, warm-up state).
- **Concurrency.** One warehouse connection, used by one engine task at a time; the agent holds it only while a tool runs, never while the model thinks.

Setting	Meaning
`CAUSA_BACKEND`	`exasol` or `duckdb`
`CAUSA_EXASOL_DSN`, `_USER`, `_PASSWORD`, `_ENCRYPTION`	Exasol connection (password: a personal access token on SaaS)
`CAUSA_FALLBACK_DUCKDB_PATH`	local copy to answer from while Exasol is away
`OPENAI_API_KEY` / `ANTHROPIC_API_KEY`	the agent's model (Ask tab, `causa ask`)
`CAUSA_AGENT_PROVIDER`, `CAUSA_OPENAI_MODEL`	force a provider; pick an OpenAI model (default `gpt-5-nano`)
`CAUSA_SLACK_WEBHOOK_URL`	where alerts and briefs are posted
`CAUSA_WATCH_MINUTES`	how often the server looks for new serious incidents (0 = off)
`CAUSA_BRIEF_AT`, `CAUSA_BRIEF_METRIC`	daily brief time (`HH:MM`) and metric

How it works

The simulator (`causa.simulator`)

A B2B reseller with four regional order feeds, FX conversion, a RAW → STG → MART → RPT pipeline, and realistic consumers: Tableau/Looker dashboards, a Monday board pack, an ML churn model whose scores are synced to Salesforce, a month-end finance close, a stale nightly extract nobody reads, and ad-hoc analysts.

The simulator generates what *actually* happened (ground truth, `SIM.*`) separately from what *landed* in the warehouse. Business events and data incidents are injected on top:

Business (real)	Data (incidents)
Market launch, promotion, price change, regional slowdown, whale churn	Schema drift that silently drops a feed, duplicate load, wrong FX rate, null join keys, late arrival, failed job + manual re-run

Every statement is written to `OPS.QUERY_LOG`, shaped like Exasol's `EXA_DBA_AUDIT_SQL`, with the client strings real tools send. Causa never reads `SIM.*`; it is only used to score Causa.

Engine 1: lineage (`causa.lineage`)

Built purely from the query log, with no instrumentation:

- statements are fingerprinted (vectorised in Polars), so each template is parsed once
- `sqlglot` resolves every projection to its source columns through CTEs, subqueries and views
- **gating columns** (JOIN / WHERE / GROUP BY) are tracked separately from derived columns, because that is how rows silently vanish (null join keys, a renamed timestamp column)
- actors (connectors, pipelines, dashboards, reports, reverse-ETL syncs, analysts) are identified from session client strings
- `impact()` computes the blast radius of a bad table over a time window

Engine 2: data health (`causa.health`)

Zero-configuration monitoring that uses lineage to decide *what* matters:

- **discovery**: load timestamp, segment column (feed, currency…), event time, and the smallest key that is unique on almost every load day (so a duplicate load cannot hide its own key)
- **columns watched**: keys plus every column that gates rows downstream, directly or through columns derived from it; numeric columns only if downstream values are computed from them
- **one pushed-down query per table** returns daily volume, null counts, means, lateness and duplicate keys per segment; **row conservation** is checked along every pipeline edge
- **robust detection**: trailing-only median/MAD baselines, weekday-aware volume, Poisson and small-sample binomial floors, and a minimum practical effect per signal
- **query-log checks**: schema changes (attributed to the actor), failed writes and their re-runs, new category values (new market, new currency)
- **lineage-aware de-duplication**: an anomaly echoed downstream is folded into its upstream root; related roots are grouped into **cases** whose headline is the most telling incident
- **mix-aware severity**: a shifted mean over a mixed population (e.g. discounts during a promotion) is capped at medium; the same shift in a reference table (FX rates) is critical

On the default scenario and two unseen seeds, `causa eval health` gives 100% recall on data incidents, 100% calm on business changes (none raised above medium) and 100% alert precision. The simulator is the benchmark, so these numbers say the engine works on these incident types, not that it will on every warehouse.

Engine 3: why a metric moved (`causa.metrics`)

Metrics are declared in `causa.yaml` (table, time column, summed measure, optional volume such as distinct orders, dimensions, high-cardinality "deep" dimensions such as customer).

- **recursive drill-down, pushed down**: one `GROUPING SETS` query per level returns current and baseline values for every element of every dimension; Causa drills into each driver with a new query (region -> country -> customer). A whole explanation is typically 3-8 queries.
- **choosing the dimension** (after Adtributor): elements must carry at least 10% of the change and be *over-represented* (lift >= 1.3 vs their baseline share), so big segments do not "explain" a change merely by being big; dimensions are ranked by excess explanation, so a single customer that fully explains a change beats its country and region
- **volume vs rate**: each driver's change is split exactly (two-player Shapley value) into a volume effect and a value-per-unit effect. Lost rows are pure volume; a wrong FX rate is pure rate.
- **uniform vs concentrated**: a broad segment whose every part moved by the same percentage is flagged as uniform (how lost data looks); a vanished segment is reported as "stopped entirely"
- **significance**: the change is compared with the same comparison over the previous ten windows (robust z), and the default baseline covers the same weekdays a week earlier
- **snapshots and restatements**: Causa records what every metric *showed* each night, so values that changed after they were reported (late batches, failed loads that were re-run) stay visible even after the warehouse has silently healed

Engine 4: real change or broken data? (`causa.verdict`)

`causa verdict` splits a metric's change into data incidents and real business change:

- **relevance and linkage**: a health case counts if lineage says its table feeds the metric. Its affected rows are found through a *bridge*: a table on the lineage path that carries the affected segment (feed, currency) and shares a key with the metric table. `CURRENCY=JPY` reaches the fact table through `STG.ORDERS`.
- **days**: health incidents are dated by load day, metrics by business date; the typical lag of each table (measured, not configured) converts one into the other. Incidents in the *baseline* count too: they distort what the present is compared with.
- **impact, by mechanism**, each method stated in the output:
- schema drift, lost feeds: difference-in-differences against the rest of the metric, with a 95% band from historical volatility and an alignment z-score
- wrong reference values (FX): the observed factor applied to the affected rows
- duplicated keys: observed = true / (1 - duplicate share)
- partial NULLs in a gating column: row loss, assuming dropped rows were average
- late batches that arrived, failed loads that were re-run: *healed*, no impact on today's data (the restatements show what people saw at the time)
- **evidence in log-odds**, every item printed: severity, integrity signals vs shape-only signals, a new category value (business-like), alignment, restatements. A finding counts as data at 60%.
- **no double counting**: findings about the same rows on the same days are merged
- **business remainder**: judged against normal variation and explained again by attribution on the days no incident touched (that is where a churned customer surfaces under a broken feed)

`causa eval verdict` compares the data impact with the true gap (reported change minus true change) for the week starting at every event. With estimates alone: 30 of 33 windows within 10% or inside the 95% band (the misses: partial-NULL row loss, overestimated ~60% because the dropped rows were smaller than average). With engine 5's replay: **33 of 33, worst error 0.8%**, and no business event ever claimed as data.

Engine 5: counterfactual replay and recall (`causa.replay`, `causa.recall`)

What would the metric have shown without the incident? Causa re-runs the company's **own pipeline**:

1. a **repair** is inferred from the health evidence, never configured: - duplicated keys: keep the first load of each inferred key - a column that went NULL right after an `ADD COLUMN`: verified rename, `COALESCE(old, new)` - NULL join keys: kept under an `(unknown)` member of the dimension they join (found from the parsed `ON` conditions), so the value survives and only the category is unknown - a wrong reference value: interpolated between the last good value and the next
2. the exact INSERT/MERGE statements the query log recorded on the affected load days, for every table between the damage and the metric, are re-run in the `CAUSA_REPLAY` schema with table names rewritten by sqlglot: once on the original data (**control**), once on the repaired data
3. the control run must reproduce every logged row count (proof the replay is faithful); the metric on repaired minus control is the incident's exact effect

On the simulator the replay matches the true misstatement to the euro for duplicates, NULL keys and the renamed column, and within 0.2-0.8% for the interpolated FX rate.

`causa recall` then lists everyone exposed to the wrong numbers: exposure window (until the data healed, or *still open* if the warehouse holds the damage), misstatement per day, and every dashboard, scheduled report (editions, recipients), ML pipeline, reverse-ETL sync, spreadsheet and analyst that read affected data, plus the pipeline that carried the incident and the load days to re-run. Each gets a drafted correction notice; `--out notices.md` writes them to a file.

The web UI (`causa serve`, then http://127.0.0.1:8000)

A dependency-free single page (`src/causa/web/`, plain HTML/CSS/JS, no build step, no CDN) over the API:

- **Overview**: the metric's daily history with every data incident shaded; pick a week and Causa splits its change into *broken data* and *real business change* (with a waterfall and one forwardable sentence), shows each incident's cause, replayed impact, confidence and evidence, the real business drivers on clean days, and **who saw wrong numbers**, with a ready-to-send correction notice per report, dashboard and sync.
- **Incidents**: every health case on a timeline, with its evidence, the tables it touched, its week, and its blast radius.
- **Lineage**: connectors → RAW → STG → MART → reports → consumers, reconstructed from the query log; select anything to trace it up and down, see the columns that gate its rows and its schema changes, and light up everything a broken table would hit.
- **Ask**: the investigator agent, with every tool call it made shown under its answer (needs `OPENAI_API_KEY` or `ANTHROPIC_API_KEY` in `.env`).
- **Take the tour** (or open `/?tour=1`): six stops through the story, starting at the latest critical incident, so it works on any data.

Speed on a remote warehouse. Every query is a round trip (about 0.3 s from India to Frankfurt), so Causa avoids repeating them. A replay depends on the incident, not on the week being judged, so each incident is replayed once and every week that touches it reuses the result. Verdicts are remembered per question, so judge, recall and the web UI share one run, and reopening a week is instant. Measured on Exasol SaaS: first look at a week with a new incident about 17 s, a week whose incidents were already replayed about 4.5 s, a week seen before 0 s (it was 18 to 34 s).

The investigator agent, the brief and the API

`causa ask` puts a language model in front of the engines: Claude (`claude-opus-5-5`, high effort) or OpenAI (`gpt-5-nano` by default, the cheapest OpenAI model with function calling). The model plans the investigation; **every figure comes from a tool**:

Tool	Engine
`judge_change`	verdict: data incidents (replayed) vs business change, evidence, restatements
`explain_metric`	attribution drill-down with volume/rate split and significance
`health_cases`	data health cases in a date range
`recall_consumers`	exposure, misstatement per day, consumers and draft notices
`lineage`	upstream/downstream, loaders, row-gating columns, schema changes
`run_sql`	one read-only SELECT (parsed by sqlglot; rows capped; ground-truth schema refused)

Tools are strict JSON schemas; system prompt and tools are prompt-cached; the conversation is append-only; refusals fall back server-side (`fallbacks: "default"`); every tool call is recorded and shown, so each answer has an audit trail. The loop is tested against a scripted client.

Choosing the model. Put `ANTHROPIC_API_KEY` or `OPENAI_API_KEY` in `.env`. With both, Claude is used unless `CAUSA_AGENT_PROVIDER=openai`; `CAUSA_OPENAI_MODEL` picks another OpenAI model. Both agents share the toolbox, the prompt and the audit trail; the OpenAI one runs on the Responses API (function calling with the same strict schemas, `store=False`, reasoning passed back encrypted). Measured on Exasol SaaS with `gpt-5-nano`: a "why did revenue drop, is it real, who saw it?" question takes 3 tool calls and about 13k input / 8k output tokens, under half a US cent.

`causa brief` is the morning message. It is **deterministic** (no language model): last 7 days vs the same weekdays before, the data/business split, each incident with its replayed impact and where it was seen, the real business movement on unaffected days, and new incidents since yesterday. It renders as text or Slack Block Kit and posts only with `--post` and a configured `CAUSA_SLACK_WEBHOOK_URL`.

`causa serve` exposes the same through FastAPI: `/metrics`, `/metrics/{m}/explain|verdict|recall|brief`, `/health/cases`, `/lineage/{table}` and `POST /ask` (multi-turn by `conversation_id`).

Layout

```
src/causa/
  warehouse/     Exasol (pyexasol) and DuckDB backends, failover between them, metering
  sources/       query-log sources (Exasol audit log, table) and the catalog
  lineage/       statement parser, actor identification, lineage graph
  health/        monitor discovery, profiling SQL, detectors, cases
  metrics/       metric definitions, attribution, narrative, snapshots and restatements
  verdict/       linkage, impact estimators, evidence and verdicts
  replay/        repair inference and counterfactual replay of the pipeline
  recall.py      exposure, consumers and correction notices
  agent/         investigator agent (Claude or OpenAI tool-use loop), its tools, pricing
  brief.py       deterministic morning brief (text and Slack Block Kit); slack.py posts it
  api.py         FastAPI service; workspace.py caches what the engines share
  mcp_server.py  the agent's tools over MCP (stdio), for Claude Desktop and Claude Code
  watch.py       new-incident alerts and the daily brief, recorded in CAUSA.NOTIFICATIONS
  onboarding.py  causa init: metric definitions proposed from the catalog
  benchmark.py   randomly drawn companies, scored in parallel (causa eval bench)
  store.py       Causa's own tables (CAUSA schema)
  evaluation.py  scoring against the simulator's ground truth
  simulator/     synthetic company, scenario, incidents, workloads
  cli.py
tests/
docs/            problem statement, deployment guide, benchmark, project write-up, demo
script
docker/          docker-compose for a local Exasol (x86_64)
Dockerfile       the production image (built by CI only)
LICENSE          MIT
```

Licence

MIT, see LICENSE. Copyright (c) 2026 Shweta Rani.

Deployment and run guide

Causa is one Python package with one entry point, `causa`. The same image or checkout serves the web UI, the HTTP API, the MCP server and the CLI. Configuration comes from `CAUSA_*` environment variables or a `.env` file in the working directory (`src/causa/config.py`); `.env.example` lists them all.

1. Local run, no Exasol needed

Requirements: Python 3.12+ and uv.

```
git clone https://github.com/rani700/causa.git && cd causa
uv sync
uv run causa sim run      # builds the demo company in data/causa.duckdb (~70 s)
uv run causa serve        # web UI at http://127.0.0.1:8000, API docs at /docs
```

With no `.env`, the backend is DuckDB (`CAUSA_BACKEND=duckdb`, file `data/causa.duckdb`). All SQL is written in the Exasol dialect and transpiled for DuckDB, so this is the same engine code that runs on Exasol. Open http://127.0.0.1:8000/?tour=1 for the guided tour.

Checks:

```
make test                 # uv run pytest -q
make lint                 # ruff, ruff format --check, mypy
```

2. Running against Exasol

Exasol SaaS

1. Create a database (free trial at https://cloud.exasol.com/signup).
2. Add the IPv4 address Causa connects from to **Security → allow list**.
3. From **Connect via tools**, take the connection string, the user name and a personal access token.
4. Configure Causa:

```
cp .env.example .env
```

Then edit `.env`:

Variable	Value
`CAUSA_BACKEND`	`exasol`
`CAUSA_EXASOL_DSN`	`host:8563` from the connection string
`CAUSA_EXASOL_USER`	the database user
`CAUSA_EXASOL_PASSWORD`	the personal access token
`CAUSA_EXASOL_ENCRYPTION`	`true`
`CAUSA_EXASOL_TLS_INSECURE`	remove the line or set `false`; `true` is only for the local docker-db's self-signed certificate

Never commit `.env` (it is in `.gitignore`).

Local Exasol in Docker

```
make exasol-up            # docker compose -f docker/docker-compose.yml up -d
make exasol-down
```

The `exasol/docker-db` image is x86_64 only; on Apple Silicon it runs under emulation and needs about 8 GB of memory for Docker. The defaults in `.env.example` (`localhost:8563`, TLS check off) are meant for this local database. Set `EXASOL_IMAGE` to pin a specific image tag.

Demo data on Exasol

```
uv run causa sim run                 # builds the simulated company in Exasol (~35 min over
a long link)
uv run causa eval health && uv run causa eval verdict
uv run causa serve
```

Your own warehouse

```
uv run causa init                    # proposes causa.yaml (metrics, dimensions), checks
the audit log
uv run causa serve --query-log audit  # lineage from EXA_STATISTICS.EXA_DBA_AUDIT_SQL
```

`--query-log` also takes a `SCHEMA.TABLE` holding a query log; the default, `OPS.QUERY_LOG`, is the simulator's.

Database permissions

Causa needs a user that can:

- **read the tables behind the metrics** declared in `causa.yaml`, and the pipeline tables upstream of them (health checks and replay read them);
- **read Exasol's audit log** (`EXA_STATISTICS.EXA_DBA_AUDIT_SQL` and `EXA_DBA_AUDIT_SESSIONS`): this needs `SELECT ANY DICTIONARY`, and auditing must be enabled (it is on by default on SaaS); `causa init` reports if it cannot read the log;

- **create and write its own two schemas**: `CAUSA` (metric snapshots, notifications) and `CAUSA_REPLAY` (scratch tables of a replay, dropped after each run).

It never writes to the company's own schemas, and nothing in the pipeline has to change.

3. Docker image

The `Dockerfile` builds a two-stage image: dependencies and the package from `uv.lock`, plus the demo company built into `/app/data/causa.duckdb` as the fallback copy; the runtime stage runs as a non-root user and serves on port 8000 (`causa serve --host 0.0.0.0 --port 8000`).

```
docker build -t causa .
docker run --rm -p 8000:8000 causa                    # built-in DuckDB copy
docker run --rm -p 8000:8000 --env-file .env causa    # Exasol, from your .env
```

The image sets `CAUSA_FALLBACK_DUCKDB_PATH=/app/data/causa.duckdb`, so when Exasol is configured the built-in copy is already in place as the failover.

4. Production notes

- **Probes.** `GET /health` is liveness (the process answers, no dependency checks). `GET /ready` is readiness: which warehouse is answering (Exasol or the fallback copy) and the warm-up state. It never queries the warehouse.
- **Failover.** With `CAUSA_FALLBACK_DUCKDB_PATH` set, a lost Exasol connection (never an SQL error) switches to the local DuckDB copy, retried every two minutes; the UI says so. This keeps the demo up while an idle SaaS cluster has stopped itself.
- **Warm-up.** `causa serve` prepares every incident's investigation in the background at start (`--no-warm` to skip). Pages never wait behind it.
- **Fresh data.** Every five minutes the server checks the query log's newest statement; new activity reloads lineage and health and drops cached answers.
- **Watcher (optional).** `CAUSA_WATCH_MINUTES` makes the server look for new serious incidents at that interval (default `0`, off). Each is announced once and recorded in `CAUSA.NOTIFICATIONS` (`GET /notifications`). `CAUSA_BRIEF_AT` (`HH:MM`, server local time) and `CAUSA_BRIEF_METRIC` add a daily brief. Without a server, `causa watch` runs the same loop (`--once` for cron).
- **Slack (optional).** `CAUSA_SLACK_WEBHOOK_URL` sends alerts and briefs to Slack. Without it they are only recorded.
- **Ask tab (optional).** `ANTHROPIC_API_KEY` or `OPENAI_API_KEY` enables the investigator agent. With both, Claude is used unless `CAUSA_AGENT_PROVIDER=openai`; `CAUSA_OPENAI_MODEL` picks the OpenAI model (default `gpt-5-nano`). Everything else works without a key.
- **Concurrency.** One warehouse connection per process, used by one engine task at a time.
- **Secrets.** Pass the Exasol token, model keys and the Slack webhook as environment variables from a secret store, never baked into the image.

5. Kubernetes and GitOps

The live demo runs this way:

1. **Build.** `.github/workflows/release-and-publish.yml` runs on every push to `main`: it cuts the next patch release (`v0.0.1`, `v0.0.2`, …), builds the `Dockerfile` for `linux/amd64` and pushes `ghcr.io/rani700/causa` tagged with that version, `latest` and the short commit SHA. `.github/workflows/ci.yml` runs lint, types and tests on every push and pull request.
2. **Deploy.** Argo CD watches a GitOps repository holding the Kubernetes manifests and syncs them to the cluster. A minimal setup is: - a Deployment running the image on port 8000, with a liveness probe on `/health` and a readiness probe on `/ready`; - a Secret with `CAUSA_BACKEND`, the `CAUSA_EXASOL_*` connection settings and any optional keys, exposed to the container as environment variables; - a Service and an Ingress with TLS for the public hostname.
3. **Roll out.** A new image is picked up by updating the tag in the GitOps repository (or by restarting the Deployment when it tracks `latest`).

Licence

MIT License

Copyright (c) 2026 Shweta Rani

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.